

API Project Analysis Guide

Step-by-step backend walkthrough for the .NET API project that powers the Angular storefront. This guide explains the architecture, request pipeline, controllers, services, data access, authentication, payments, caching, AI/RAG features, and interview-ready explanations.

Platform: ASP.NET Core 5 Web API

Architecture: API + Core + Infrastructure

Data: EF Core + PostgreSQL

Auth: ASP.NET Identity + JWT

AI: Azure AI Search + OpenAI + RAG

How To Read This Document

1. Start with the overview and architecture diagram.
2. Study the request pipeline to understand how a call travels through the API.
3. Read controllers and services together so business flow makes sense.
4. Finish with the interview section and learning plan.

Easy mindset: think of **API** as the presentation layer, **Core** as business contracts and domain models, and **Infrastructure** as data access and external integrations.

1. Application Overview

This backend is the server side of the e-commerce application. It exposes endpoints for products, baskets, authentication, addresses, orders, delivery methods, Stripe payments, and AI-assisted Q&A.

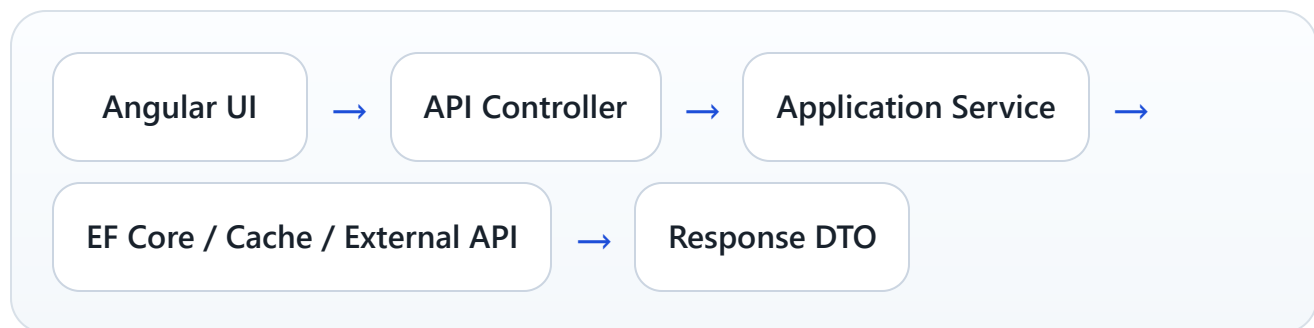
Main Backend Responsibilities

- Serve product catalog data
- Manage customer baskets
- Handle login, registration, and address storage
- Create orders and track order history
- Integrate with Stripe for payment intents
- Support AI chat using Azure AI Search and OpenAI

Likely Consumers

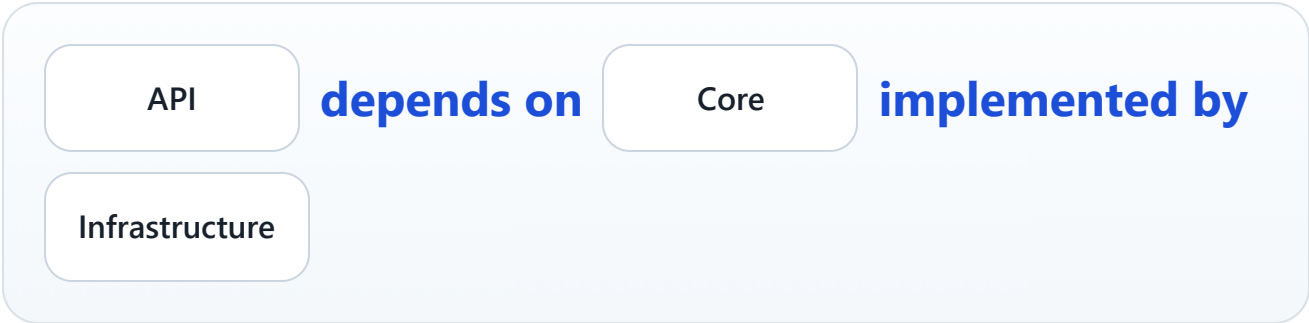
- The Angular storefront
- Local Swagger testing
- AI indexing/sync commands at startup

Main Business Flow



2. Solution Structure

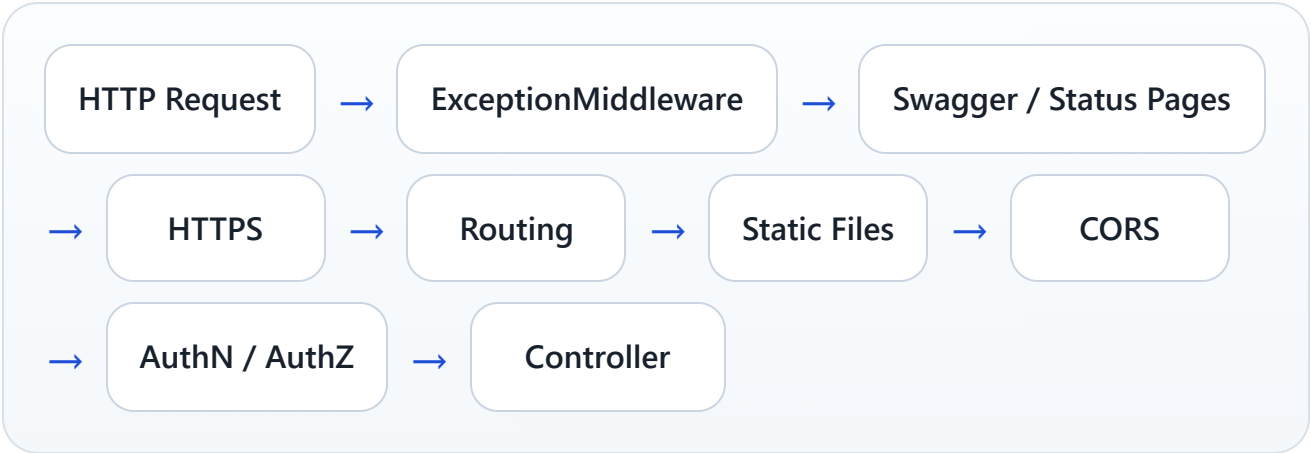
Project / Folder	Purpose	Easy Analogy
API	HTTP entry point, controllers, middleware, startup, DTOs	Presentation layer / web host
Core	Entities, interfaces, domain models, specifications	Business contracts and domain language
Infrastructure	EF Core, repositories, services, identity, Stripe, AI integrations	Data and external systems layer
Controllers	Expose REST endpoints	Traffic desk receiving requests
Dtos	Request/response contracts sent over HTTP	Clean API-facing models
Middleware	Cross-cutting request handling	Pipeline checkpoints
Extensions	Service registration and startup helpers	Organized startup wiring
Data / Identity	DbContexts, seeding, repositories, EF config	Persistence layer



3. Startup And Request Pipeline

Program.cs builds the host, optionally runs migrations, and can initialize Azure AI Search or sync products/RAG documents based on command-line flags or environment variables.

Request Pipeline



Pipeline Part	What It Does
ExceptionMiddleware	Catches unhandled exceptions and returns JSON error responses
UseStatusCodePagesWithReExecute	Maps status codes like 404 to a clean error controller response
UseStaticFiles	Serves Angular build output and content files
UseCors	Allows configured frontend origins
UseAuthentication	Validates JWT bearer tokens
UseAuthorization	Applies [Authorize] rules
MapFallbackToController	Returns Angular index.html for SPA routes

4. Controllers And Endpoints

Controller	Main Endpoints	Business Role
ProductsController	GET /products, /products/{id}, /products/brands, /products/types	Catalog browsing
BasketController	GET /basket, POST /basket, DELETE /basket	Temporary basket persistence
AccountController	/account/login, /account/register, /account, /account/address	Authentication and profile data
OrdersController	POST /orders, GET /orders, GET /orders/{id}, /orders/deliveryMethods	Order creation and history
PaymentsController	POST /payments/{basketId}, GET /payments/config, POST /payments/webhook	Stripe payment workflow
AiController	GET /ai/documents, POST /ai/chat	AI search and answer generation

Controllers in this project are fairly thin. Most business logic sits in services, which is a good architectural pattern.

5. Data Access And Persistence

The API uses **EF Core** with two PostgreSQL-backed contexts:

- `StoreContext` for products, orders, delivery methods
- `AppIdentityDbContext` for ASP.NET Identity users

Main Data Patterns

Direct DbContext Usage

Used in services like `ProductService` and `OrderService` for straightforward queries.

Repository + Unit of Work

Used especially in payments and generic data access where reusable patterns help.

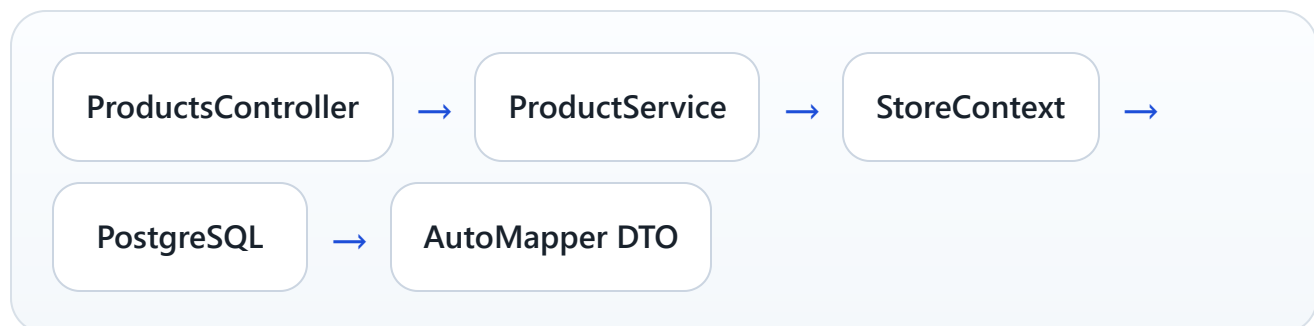
Basket Repository

Basket data is not stored in the main relational tables. It uses in-memory cache.

AutoMapper

Maps entities to DTOs and keeps controllers from exposing persistence models directly.

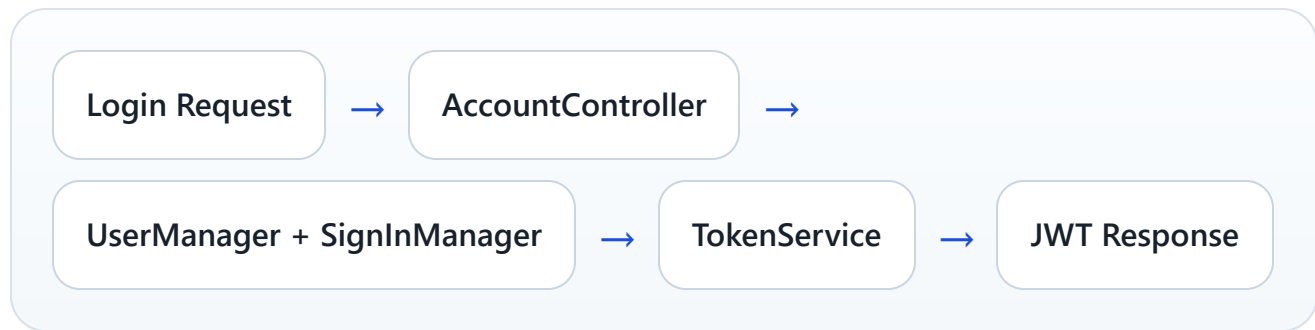
Typical Product Read Flow



6. Business Services

Service	Purpose	Key Logic
ProductService	Catalog reads	Filtering, sorting, paging with EF Core
OrderService	Order creation/history	Converts basket items into order items and links payment intent
PaymentService	Stripe integration	Creates or updates payment intents, synchronizes basket totals, updates order status from webhook results
TokenService	JWT generation	Creates signed tokens for authenticated users
AiChatService	AI orchestration	Intent detection, retrieval gating, source validation, answer generation
AiRetrieverService	Vector retrieval	Generates embeddings, queries Azure AI Search, returns ranked matches
AzureAiSearchService	Index management and sync	Creates indexes and uploads products/RAG docs in batches

7. Authentication And Authorization



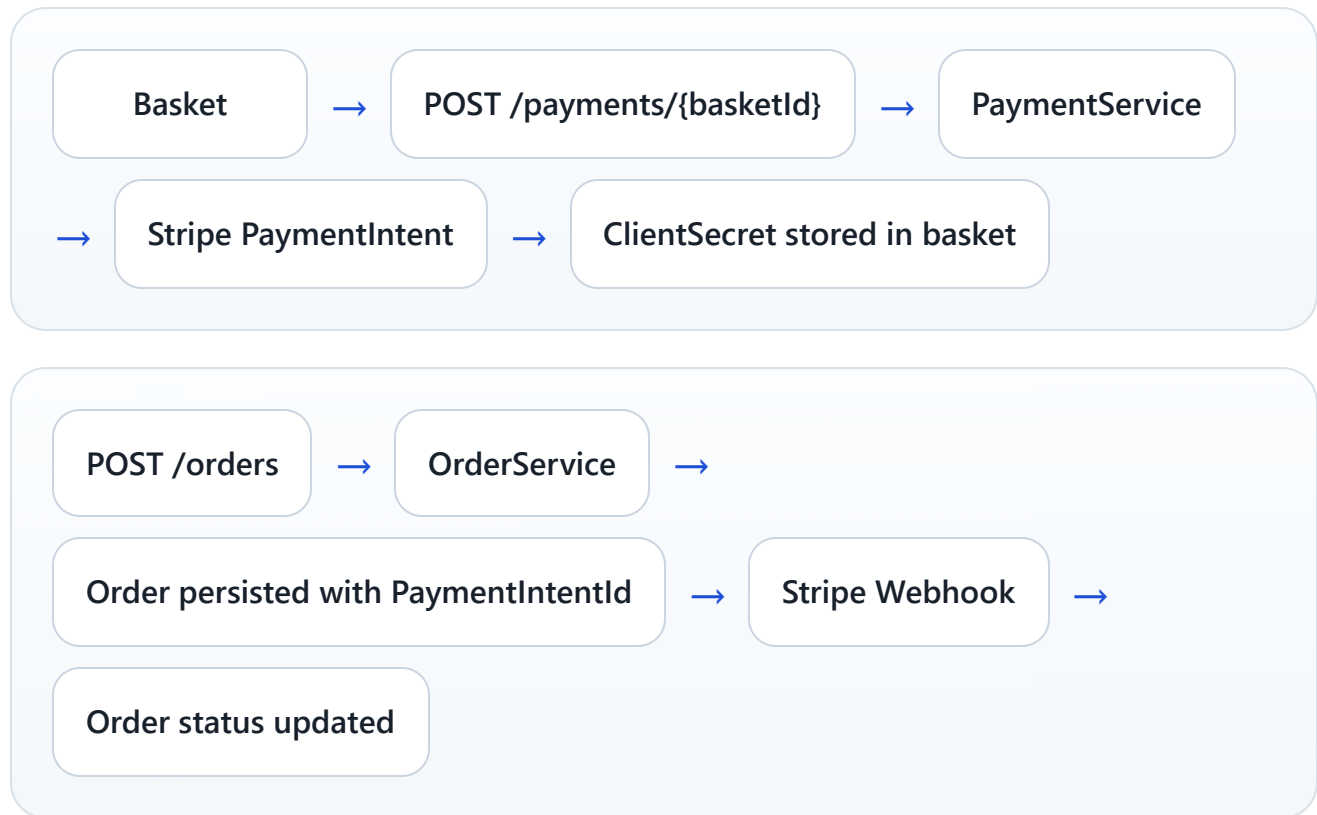
- Uses ASP.NET Core Identity with AppUser stored in AppIdentityDbContext.
- Uses JWT Bearer authentication.
- Protected endpoints use [Authorize] on controllers or actions.
- User email is retrieved from claims using a small extension method.

Interview-friendly explanation

"The API authenticates users with ASP.NET Identity and issues JWTs. Protected endpoints rely on bearer token authentication, and business logic uses the authenticated email claim to scope user data like orders and addresses."

8. Payments And Checkout Flow

The payment flow is split clearly between order creation and Stripe payment intent creation.



Good design point: the webhook updates the final payment status, so the API does not rely only on the browser saying "payment succeeded."

9. Caching

Caching Area	How It Works	Where Used
Response cache	CachedAttribute builds a cache key from request path + query and stores serialized JSON in IMemoryCache	Products and lookup endpoints
Basket cache	BasketRepository stores serialized basket JSON in memory for 30 days	Basket operations
AI embedding cache	In-memory cache for computed document embeddings	Retriever service internal optimization

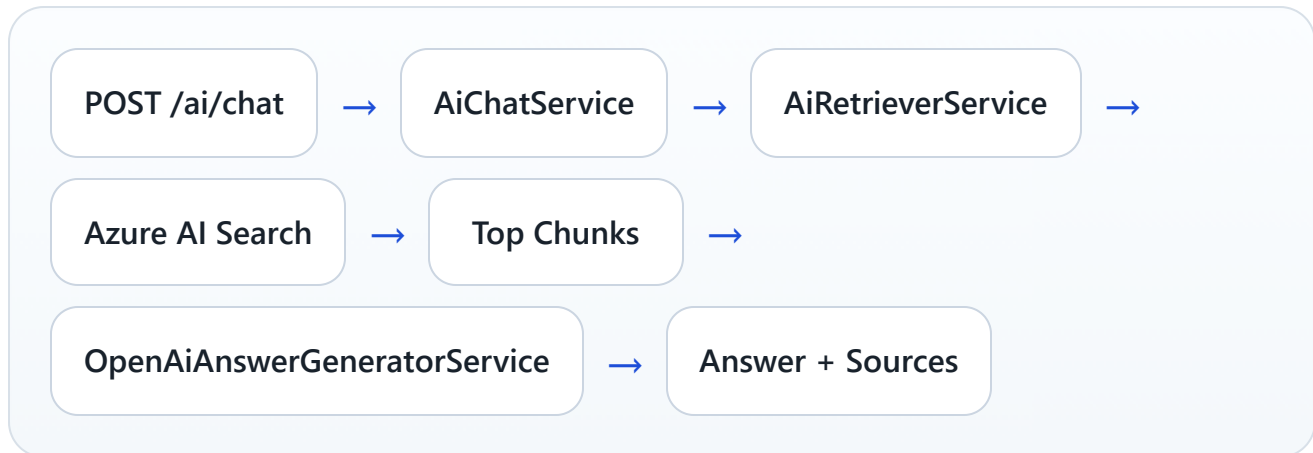
Interview-friendly explanation

“The API uses in-memory caching in three different ways: HTTP response caching for product reads, basket persistence for cart state, and AI embedding caching to reduce repeated vector preparation work.”

10. AI And RAG Architecture

This is one of the most interesting parts of the project. The API is not just CRUD-based. It also supports a retrieval-augmented question answering flow.

AI Request Flow



What Makes It Senior-Level

- Intent detection distinguishes product questions from policy/delivery questions.
- Price constraints can be detected from natural language like "under 100".
- Confidence thresholds are used to avoid weak answers.
- Source coverage checks reduce hallucination risk.
- Embedding generation falls back locally if OpenAI is not configured.
- Azure AI Search indexing is separated from query-time answer generation.

11. Error Handling And Logging

Mechanism	What It Does	Assessment
ExceptionHandler	Catches unhandled exceptions and returns JSON error payloads	Strong centralized design
ExceptionHandler	Produces consistent status-code-based error responses	Good for 404/other status code handling
NLog	Structured server-side file logging	Better than plain console-only logging
Model validation response factory	Converts invalid model state into a clean validation error format	Good API consumer experience

12. Security Review

- Good: JWT authentication is configured correctly at a high level.
- Good: protected business endpoints use [Authorize].
- Good: Stripe publishable vs secret key responsibilities are separated.
- Risk: appsettings.json and appsettings.Development.json currently contain development secrets and connection strings directly in the repo.
- Risk: CORS defaults are permissive for local development, which is okay for dev but should stay locked down in production.

Strong interview note: "One improvement I would prioritize is moving development secrets and tokens fully into user secrets or environment variables so they are not committed in config files."

13. Performance And Scalability

Good Choices Already Present

- Thin controllers
- Read caching for products
- Batch indexing for Azure AI Search
- Async EF Core and async I/O throughout
- OpenAI retry logic

Possible Improvements

- Replace in-memory basket store with Redis for multi-instance deployment
- Add health checks and observability endpoints
- Introduce stricter cache invalidation strategy if write traffic grows
- Reduce mixed repository/direct DbContext style if consistency matters

14. Interview Explanation Pack

Tell me about this API project.

"It is an ASP.NET Core 5 backend for an e-commerce platform. It handles catalog reads, baskets, authentication, orders, Stripe payments, and an AI-assisted retrieval flow using Azure AI Search and OpenAI."

How is the backend structured?

"The solution separates concerns into API, Core, and Infrastructure. API handles HTTP concerns, Core defines domain contracts and entities, and Infrastructure implements data access and external integrations."

How do requests flow through the system?

"A request enters the ASP.NET pipeline, passes through exception handling, routing, CORS, and authentication, then reaches a controller. The controller calls a service, which interacts with EF Core, cache, Stripe, or AI/search services before returning a DTO."

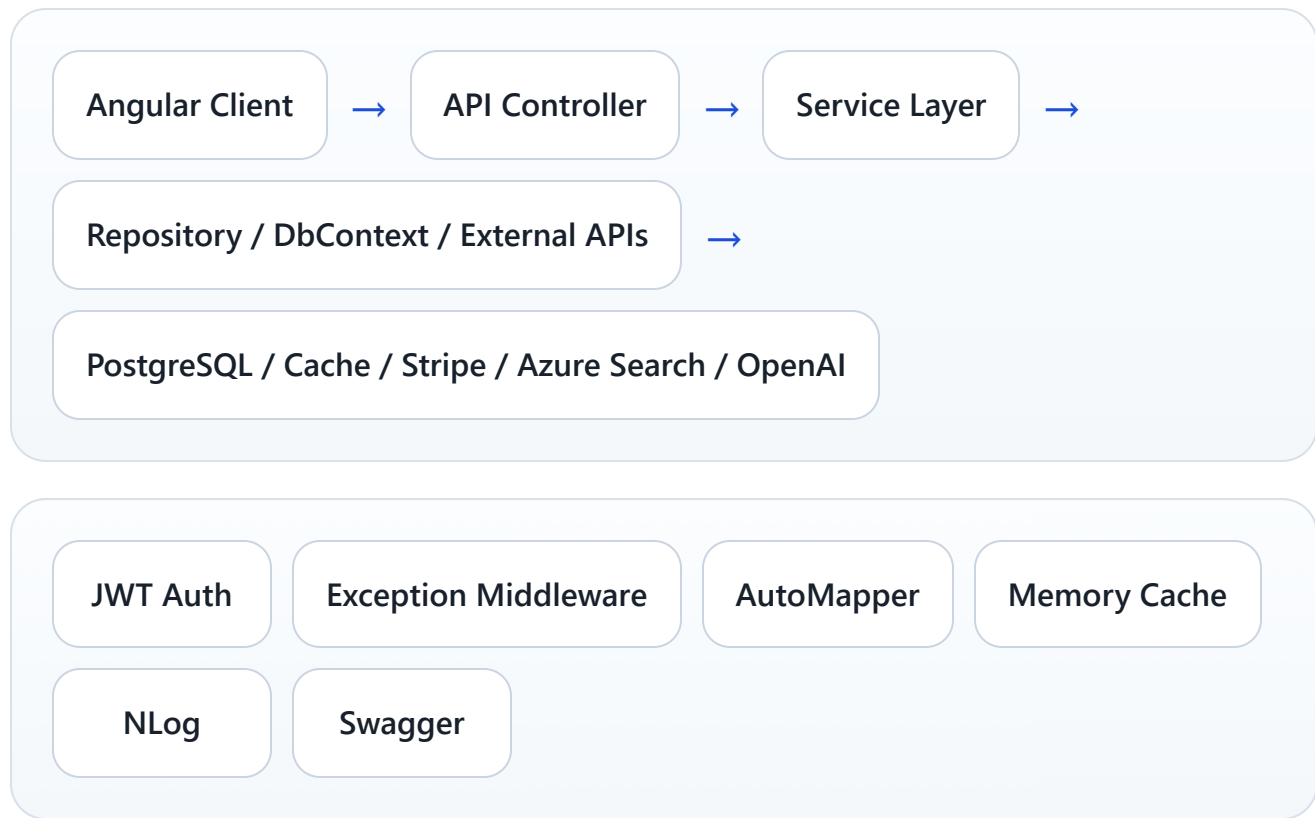
How do you handle security?

"Authentication uses ASP.NET Identity and JWT bearer tokens. Authorization is enforced with Authorize attributes, and user-specific operations are scoped using claims such as the authenticated email."

What would you improve?

"I would externalize all secrets, replace memory-only basket storage with Redis for scale-out deployments, and add richer observability such as health checks, metrics, and correlation tracing."

15. Architecture Diagram



16. Learning Plan

Best File Order To Study

1. Program.cs and Startup.cs
2. ApplicationServiceExtensions.cs and IdentityServiceExtensions.cs
3. Controllers: products, account, orders, payments, AI
4. Services: product, order, payment, AI chat/retriever
5. Contexts, repositories, AutoMapper profiles
6. Exception middleware, caching attribute, logging config

Key Concepts To Learn From This API

- ASP.NET Core request pipeline
- Dependency injection and service lifetimes
- JWT authentication with ASP.NET Identity
- EF Core querying, paging, and includes
- DTO mapping with AutoMapper
- Custom middleware
- Action filters for response caching
- Webhook processing
- AI retrieval and answer generation architecture

If you understand those files and concepts, you'll be able to explain this backend like a senior developer instead of just listing controllers.

Prepared from the API, Core, and Infrastructure projects in this solution for backend understanding and interview preparation.